SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	**Course Title:**	Computer Architecture
CO Assessed:	CO2	**Assessment:**	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	D. Srujana Bai	**Reg.No.:**	192571067
Department:	CS & BIOSCIENCE	**Date:**	

80%

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I D. Srujana Rao (192571069) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)	
Q1	3	3	4	2	3	15
Q2	4	3	3	2	3	15
Q3	4	4	4	3	2	17
Q4	4	3	3	4	2	16
Q5	4	4	2	4	3	17

80

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately(4)	Understands problem with minor gaps(3)	Partial understanding; misses key elements(2)	Misinterprets or unable to understand problem(1)
Method Selection	20%	Selects most appropriate method/formula with strong justification(4)	Selects correct method with minor errors(3)	Inappropriate or partially correct method(2)	Unable to select method(1)
Solution Process	25%	Logical, systematic steps with correct approach throughout(5)	Mostly correct steps with minor mistakes(4)	Disorganized steps; multiple errors(3)	No clear process(0)
Accuracy of Calculation	20%	All calculations accurate; correct final answer(4)	Minor calculation errors(3)	Frequent errors; incorrect final answer(2)	No valid calculations(0)
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance(3)	Basic interpretation with minor omissions(2)	Limited or unclear interpretation(1)	No interpretation(0)

1.A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

A).

Problem Understanding

The processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. The given numbers are:

- +45
- –23

The task is to perform both addition and subtraction, analyze the binary representation, check overflow, and interpret the result.

Method Selection

2's complement arithmetic is used because it allows both positive and negative numbers to be represented and processed using the same binary adder circuitry.

Solution Process:

Step 1: Convert +45 to 8-bit Binary

$45_{10} = 00101101_2$

Step 2: Convert –23 to 8-bit Binary

$23_{10} = 00010111_2$

Find 2's complement:

1's complement = 11101000

Add 1:

11101001

Therefore,

$-23 = 11101001_2$

Addition: (+45) + (–23)

00101101

\+ 11101001

00010110

Result := 00010110_2

= 22_{10}

Overflow Check

- Operands have different signs.
- Overflow occurs only when two numbers of the same sign are added and produce a result of opposite sign.
- No Overflow Occurs.

Interpretation

(+45) + (–23) = +22

The processor interprets the MSB as 0, indicating a positive result.

Subtraction: (+45) – (–23)

Subtraction is performed as:

45 + (2's complement of –23)

Since subtracting a negative number is equivalent to addition:

45 + 23

00101101

\+ 00010111

01000100

Result = 01000100_2

= 68_{10}

Overflow Check

- Both operands are positive.
- Result is also positive.
- No Overflow Occurs.

Interpretation

(+45) – (–23) = +68

The processor correctly interprets the result because the MSB is 0.

Final Result

Addition:

(+45) + (–23) = +22

Subtraction:

(+45) – (–23) = +68

No overflow occurs in either operation.

2.A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors

Problem Understanding

A Carry Look-Ahead Adder improves speed by computing carry bits in advance rather than waiting for carry propagation through each stage as in a Ripple Carry Adder.

Consider:

A = 1010

B = 0110

Method Selection

CLA uses Generate (G) and Propagate (P) signals to calculate carries simultaneously.

Generate:

Gi = Ai × Bi

Propagate:

Pi = Ai + Bi

Bit-wise Analysis

Bit	A	B	Generate (G)	Propagate (P)
0	0	0	0	0
1	1	1	1	1

2	0	1	0	1
3	1	0	0	1

Assume Initial Carry:

$C0 = 0$

Carry Equations

$C1 = G0 + P0C0$

$C1 = 0$

$C2 = G1 + P1C1$

$C2 = 1 + (1 \times 0)$

$C2 = 1$

$C3 = G2 + P2C2$

$C3 = 0 + (1 \times 1)$

$C3 = 1$

$C4 = G3 + P3C3$

$C4 = 0 + (1 \times 1)$

$C4 = 1$

Sum Calculation

$Si = Ai \oplus Bi \oplus Ci$

Result:

$1010 + 0110$

$= 10000_2$

$= 16_{10}$

Comparison with Ripple Carry Adder

Ripple Carry Adder

- Carry moves from one stage to the next.

- Delay increases with number of bits.
- Slower performance.

Carry Look-Ahead Adder

- Carry signals computed simultaneously.
- Faster operation.
- Reduces propagation delay.

Interpretation

CLA improves processor speed significantly because carry generation does not need to ripple through every stage.

Final Result

$1010_2 + 0110_2 = 10000_2$

CLA achieves faster computation than Ripple Carry Addition.

3. **In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.**

Problem Understanding

Multiply:

–6 × +5

- using Booth's Algorithm.

Method Selection

Booth's Algorithm efficiently handles signed binary multiplication and reduces the number of additions/subtractions.

Solution Process

Binary Representation

Multiplicand (M) = –6

+6 = 0110

2's complement:

1010

Therefore,

M = 1010

Multiplier (Q) = +5

Q = 0101

Initial Values

A = 0000

Q = 0101

Q-1 = 0

n = 4

Cycle 1

Q0Q-1 = 10

Perform:

A = A – M

A = 0000 – 1010

A = 0110

Arithmetic Right Shift

A Q Q-1

0110 0101 0

↓

0011 0010 1

Cycle 2

Q0Q-1 = 01

Perform:

A = A + M

1101

Shift

1101 0010 1

↓

1110 1001 0

Cycle 3

Q0Q-1 = 10

Perform:

A = A – M

0100

Shift

0100 1001 0

↓

0010 0100 1

Cycle 4

Q0Q-1 = 01

Perform:

A = A + M

1100

Shift

1100 0100 1

↓

1110 0010 0

Final Product

AQ = 11100010_2

Convert to Decimal

2’s complement of 11100010

00011110 = 30

Therefore,

Result = –30

Interpretation

Booth’s Algorithm successfully multiplies signed numbers and minimizes arithmetic operations.

Final Result

(–6) × (+5) = –30

4. **A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.**

Problem Understanding

A processor must divide two binary numbers using both Restoring and Non-Restoring Division algorithms.

Given:

Dividend = $13_{10} = 1101_2$

Divisor = $3_{10} = 0011_2$

The objective is to determine the quotient and remainder, analyze both algorithms step-by-step, and compare their efficiency

Method Selection

Two hardware division methods are used:

Restoring Division

- After subtraction, if the remainder becomes negative, the original value is restored.
- Requires additional restoration operations.

Non-Restoring Division

- Avoids restoration after every negative remainder.
- Performs either addition or subtraction in the next cycle.
- Faster and more efficient.

A. Restoring Division Algorithm

Initial Values

Dividend (Q) = 1101

Divisor (M) = 0011

Accumulator (A) = 0000

Number of bits = 4

Iteration 1

Shift Left (A,Q)

AQ = 0000 1101

After Shift:

AQ = 0001 1010

A = 0001

Subtract M

A = 0001 − 0011

A = 1110 (Negative)

Since A is negative:

Restore A

A = 1110 + 0011

A = 0001

$Q_0 = 0$

Result:

A = 0001

Q = 1010

Iteration 2

Shift Left

AQ = 0001 1010

After Shift:

AQ = 0011 0100

A = 0011

Subtract M

A = 0011 − 0011

A = 0000

Remainder is positive

$Q_0 = 1$

Result:

A = 0000

Q = 0101

Iteration 3

Shift Left

AQ = 0000 0101

After Shift:

AQ = 0000 1010

A = 0000

Subtract M

A = 0000 − 0011

A = 1101 (Negative)

Restore A

1101 + 0011

= 0000

$Q_0 = 0$

Result:

A = 0000

Q = 1010

Iteration 4

Shift Left

AQ = 0000 1010

After Shift:

AQ = 0001 0100

A = 0001

Subtract M

A = 0001 − 0011

A = 1110 (Negative)

Restore A

1110 + 0011

= 0001

$Q_0 = 0$

Result:

A = 0001

Q = 0100

Final Result of Restoring Division

Quotient (Q) = $0100_2 = 4_{10}$

Remainder (A) = $0001_2 = 1_{10}$

Therefore,

13 ÷ 3 = 4 Remainder 1

B. Non-Restoring Division Algorithm

Initial Values

Dividend (Q) = 1101

Divisor (M) = 0011

Accumulator (A) = 0000

Iteration 1

Shift Left

AQ = 0001 1010

Subtract M

0001 − 0011

= 1110 (Negative)

$Q_0 = 0$

Result:

A = 1110

Q = 1010

Iteration 2

Since A is negative, Add M

1110 + 0011

= 0001

Shift Left

AQ = 0011 0100

Subtract M

0011 − 0011

= 0000

$Q_0 = 1$

Result:

A = 0000

Q = 0101

Iteration 3

Since A is positive, Subtract M

0000 − 0011

= 1101 (Negative)

$Q_0 = 0$

Result:

A = 1101

Q = 1010

Iteration 4

Since A is negative, Add M

1101 + 0011

= 0000

Shift Left

AQ = 0001 0100

Subtract M

0001 − 0011

= 1110 (Negative)

$Q_0 = 0$

Result:

A = 1110

Q = 0100

Final Correction

Since A is negative:

A = A + M

1110 + 0011

= 0001

Final Remainder = 0001

Final Result of Non-Restoring Division

Quotient (Q) = $0100_2 = 4_{10}$

Remainder (A) = $0001_2 = 1_{10}$

Therefore,

13 ÷ 3 = 4 Remainder 1

Comparison Between Restoring and Non-Restoring Division

Feature	Restoring Division	Non-Restoring Division
Restoration Step	Required	Not Required
Number of Operations	More	Fewer
Hardware Complexity	Higher	Lower
Execution Speed	Slower	Faster
Efficiency	Moderate	High

Interpretation of Results

Both algorithms produce the same result:

13 ÷ 3 = 4 Remainder 1

However, Restoring Division requires additional restoration operations whenever a negative remainder occurs. Non-Restoring Division eliminates these extra steps by using addition in the next cycle instead of restoring immediately. This reduces hardware operations and improves execution speed.

Conclusion

Both Restoring and Non-Restoring Division correctly compute:

Quotient = 4

Remainder = 1

Among the two methods, Non-Restoring Division is preferred in modern processors because it reduces the number of arithmetic operations, decreases computation time, and improves overall system efficiency.

5.A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Problem Understanding

Represent –13.25 using IEEE 754 Single Precision and Double Precision formats.

Method Selection

IEEE 754 stores floating-point numbers in three fields:

1. Sign Bit
2. Exponent Field
3. Mantissa (Fraction)

Step 1: Convert Decimal Number

13.25

Integer Part:

$13 = 1101_2$

Fraction Part:

$0.25 \times 2 = 0.5 \rightarrow 0$

$0.5 \times 2 = 1.0 \rightarrow 1$

Therefore:

$13.25 = 1101.01_2$

Normalize:

1.10101×2^3

Single Precision (32-bit)

Sign Bit

Negative Number

S = 1

Exponent

Bias = 127

Exponent = 3 + 127

= 130

$130_{10} = 10000010_2$

Mantissa

10101000000000000000000

Final Representation

1 | 10000010 | 10101000000000000000000

Double Precision (64-bit)

Sign Bit

S = 1

Exponent

Bias = 1023

Exponent = 3 + 1023

= 1026

$1026_{10} = 10000000010_2$

Mantissa

10101000000000000000000000000000000000000000000000

Final Representation

1 | 10000000010 | 10101000000000000000000000000000000000000000000000

Floating Point Addition

Example:

13.25 + 2.5

Step 1

Convert both to normalized form.

$13.25 = 1.10101 \times 2^3$

$2.5 = 1.01 \times 2^1$

Step 2

Align Exponents

1.10101×2^3

0.01010×2^3

Step 3

Add Mantissas

1.10101

+0.01010

1.11111

Step 4

Normalize Result

1.11111×2^3

Step 5

Apply Rounding

Final Result

Single Precision:

1 | 10000010 | 10101000000000000000000

Double Precision:

1 | 10000000010 | 1010100000000000000000000000000000000000000000000000